1. Introduction to Software Engineering

Topics that will cover:

- **Definition & Need for SE**
- **Software vs Program vs System**
- **Characteristics of Software**
- **Software Crisis**
- **Software Process**
- **Software Engineering Layers**

Definition:

Software Engineering means using a proper and organized way to make software. It's not just about writing code. It is about **planning**, **designing**, **developing**, **testing**, and **taking care** of the software.

It helps create software that is:

- **Easy to use**
- **Free from bugs**
- **Fast and reliable**
- **Delivered on time**
- **Easy to fix or improve later**

Formal definition:

- ***Software Engineering*** *is the* ***systematic approach to the development, operation, and maintenance*** *of software. It applies engineering principles to software development, aiming for high-quality, reliable, and maintainable software within budget and time limits.*

In simple words:

Software engineering like **building a house**:

- You first **plan** what kind of house you need
- Then you **design** its structure
- After that, you **build** it carefully
- You **check** if everything is working (like water, lights)
- Later, you **maintain** it (repair or upgrade if needed)

Why Do We Need Software Engineering?

We need software engineering because:

- Software is becoming **bigger and more complex**
- Clients want their apps **quickly and with high quality**
- Bugs and errors can cause **huge problems**
- Sometimes need to **work in teams** and still build great software
- It saves **time and money** in the long run

Software vs Program vs System

Term	Explanation	Example
Program	A set of instructions written to perform a specific task	Calculator app
Software	A collection of **programs + data + documentation** that performs a useful function	Microsoft Word
System	A combination of **hardware and software** working together for a complete solution	ATM machine (hardware + software + OS)

- *In short:*
  Program is a part of **Software**, and **Software** is a part of a **System**.

Characteristics of Good Software

1. **Correctness:** It does what it is supposed to do.
2. **Efficiency:** Uses minimal resources (CPU, memory).
3. **Reliability:** Works well under all conditions.
4. **Maintainability:** Easy to update and fix bugs.
5. **Portability:** Can run on different systems.
6. **Usability:** Easy for users to interact with.
7. **Scalability:** Can handle increased load or growth.

What was the Software Crisis?

The **Software Crisis** refers to the **problems faced in the early days of software development**, such as:

- Projects **taking longer than expected**
- Going **over budget**
- Software had **many bugs**
- Difficult to **maintain or scale**
- Lack of **standard practices**

This led to the realization that **"coding alone isn't enough"** So, we need **engineering discipline**. That's how **Software Engineering** was born.

Software Process (SDLC)

The **Software Process** (or **Software Development Life Cycle - SDLC**) is a step-by-step method for building software.

Main Phases:

1. **Requirement Analysis** – What should the software do?
2. **Design** – How will it work internally?
3. **Implementation (Coding)** – Writing the actual code.
4. **Testing** – Checking for bugs and errors.
5. **Deployment** – Releasing the software.
6. **Maintenance** – Fixing bugs, adding features after release.

Note: A good software process ensures that the **right software is built correctly**.

Layers of Software Engineering

Software Engineering isn't just about writing code .It's a **layered discipline**:

1. **Quality Focus** – Everything is done with a focus on quality.
2. **Process Layer** – Defines the **framework and flow** of development (like SDLC).
3. **Methods Layer** – Describes **how to do tasks** (designing, coding, testing).
4. **Tools Layer** – Software tools that support each method (like IDEs, testing tools, version control).

Think of it like building a cake:

- **Quality** is the recipe standard.
- **Process** is the step-by-step procedure.
- **Methods** are the cooking techniques.
- **Tools** are your oven, spatula, and bowl!

Summary

- **Software Engineering** brings structure and discipline to software development.
- It's needed to manage growing complexity and ensure software is reliable, cost-effective, and user-friendly.
- **Software ≠ Just Code** – It includes documentation, processes, tools, and people.
- The **software process** ensures that software is built in a logical, step-by-step manner.
- Layers of SE help organize how we plan, create, and maintain software.